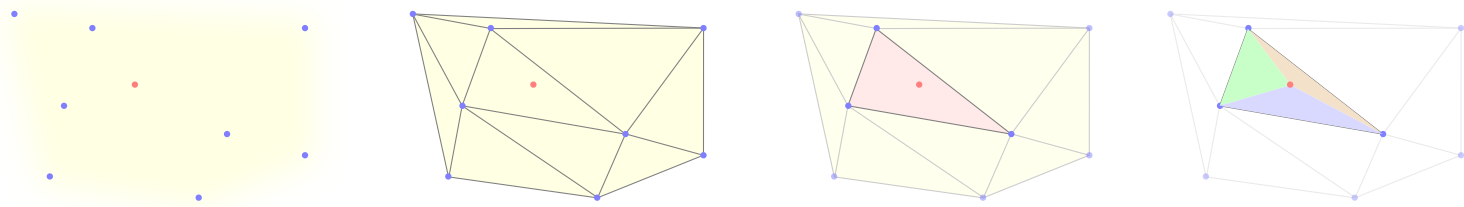
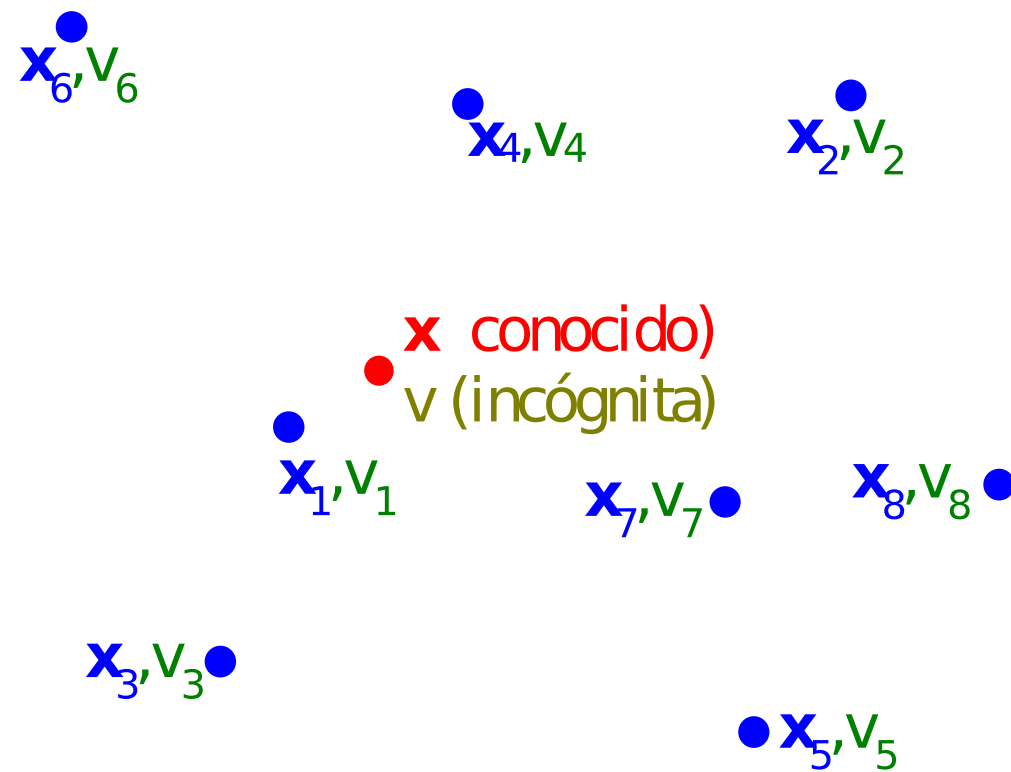


Interpolación



DEFINICIÓN DEL PROBLEMA



- Datos iniciales: Para un conjunto finito de nodos fijos
 - se conocen las posiciones y los valores asociados
- Se pretende "encontrar"
 - el valor asociado a un punto cualquiera/variable

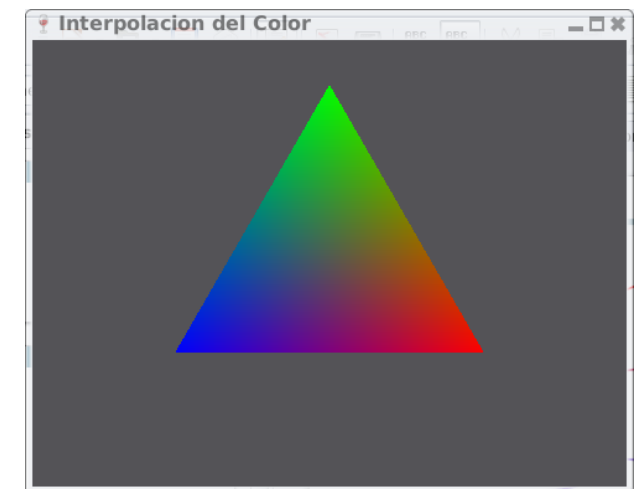
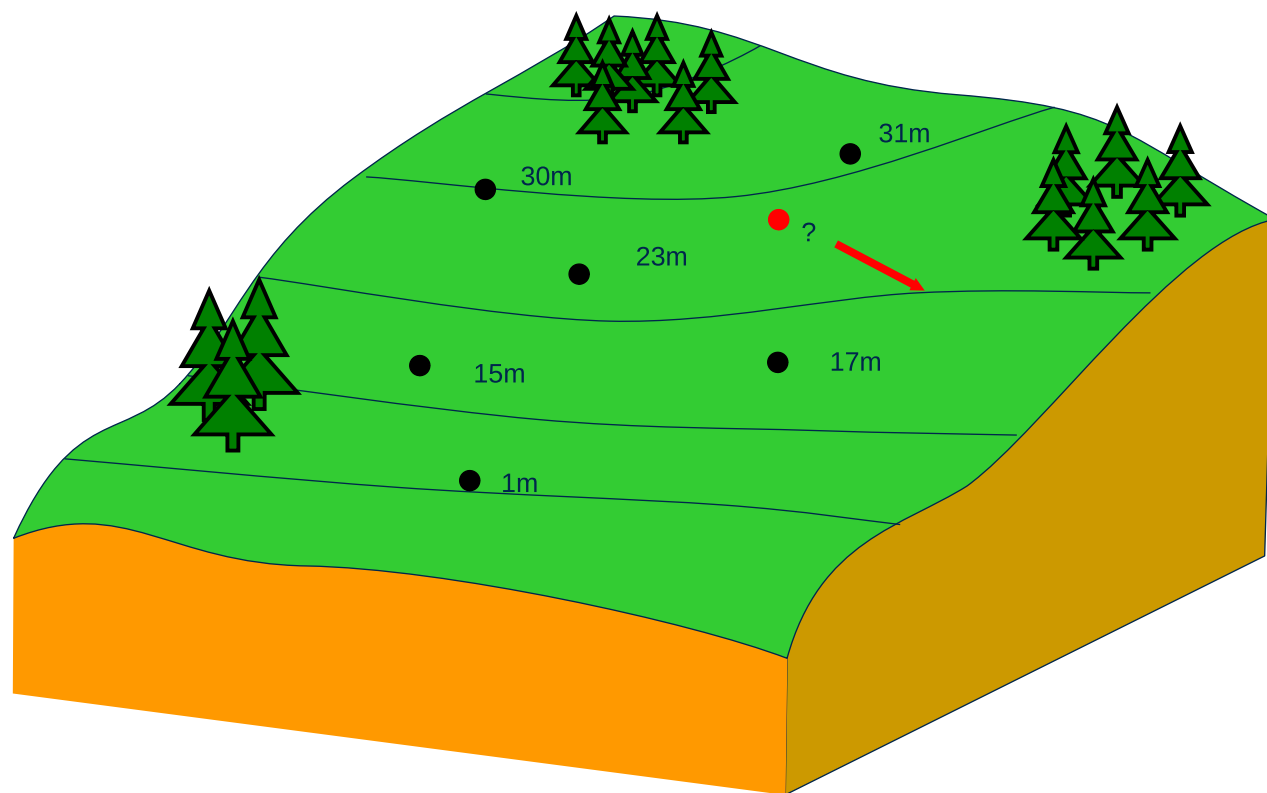
La idea básica es encontrar una forma de calcular (predecir/aproximar/inventar) un valor desconocido (ejs: color, temperatura, altura, etc) para un punto dado del espacio, a partir de valores en otros puntos que sí son conocidos.

- Los puntos conocidos son fijos, y sus valores asociados también, son datos iniciales del problema.

- El punto en el que se quiere calcular no es fijo (deberíamos poder calcular para cualquier punto del espacio, o al menos de una región donde tenga sentido hacerlo), pero sí es dato (sería el argumento) para la función de interpolación.

Por ejemplo, si tengo medida la profundidad de un río en ciertos puntos y quiero reconstruir el lecho completo, necesito alguna función que a partir de esos puntos conocidos me pueda decir cuál sería la profundidad estimada en cualquier otro punto.

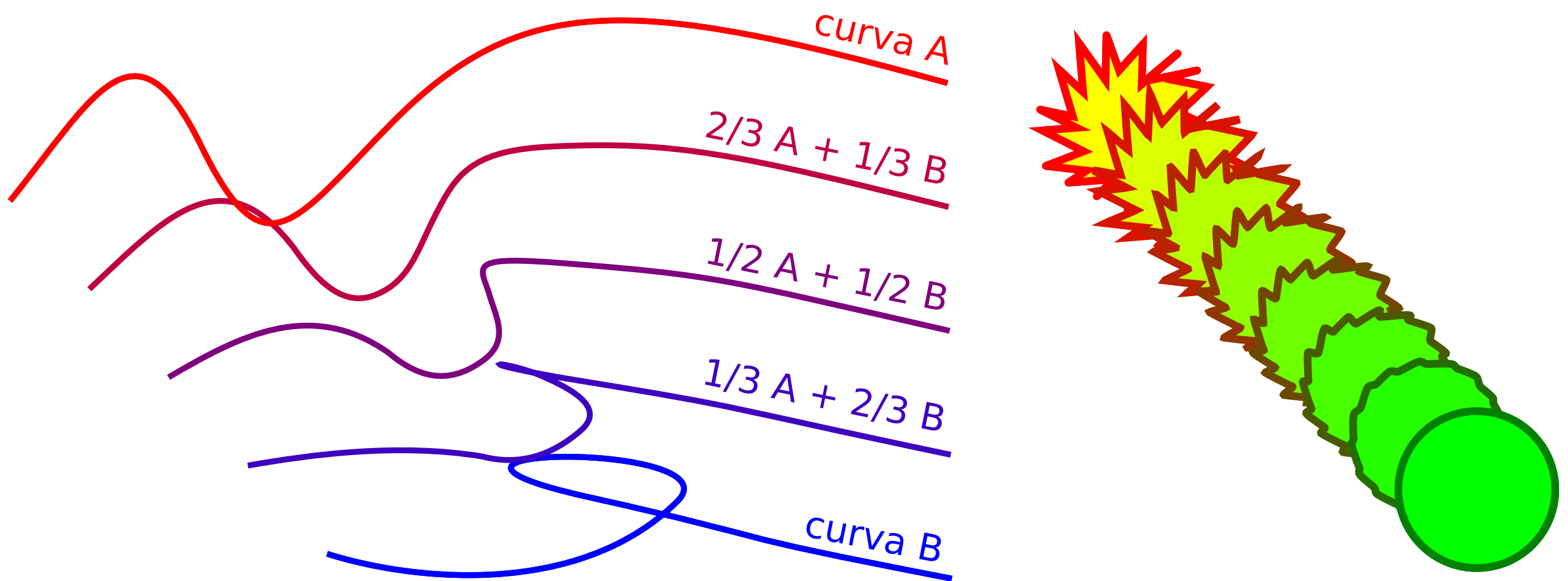
EJEMPLOS



El ejemplo de la izquierda muestra algo general (y que nos podría tocar programar a nosotros, no que nos resuelva la gpu): tengo medidas las alturas en ciertos puntos, y a partir de ellos quiero reconstruir la superficie (por ejemplo, para graficar isolineas).

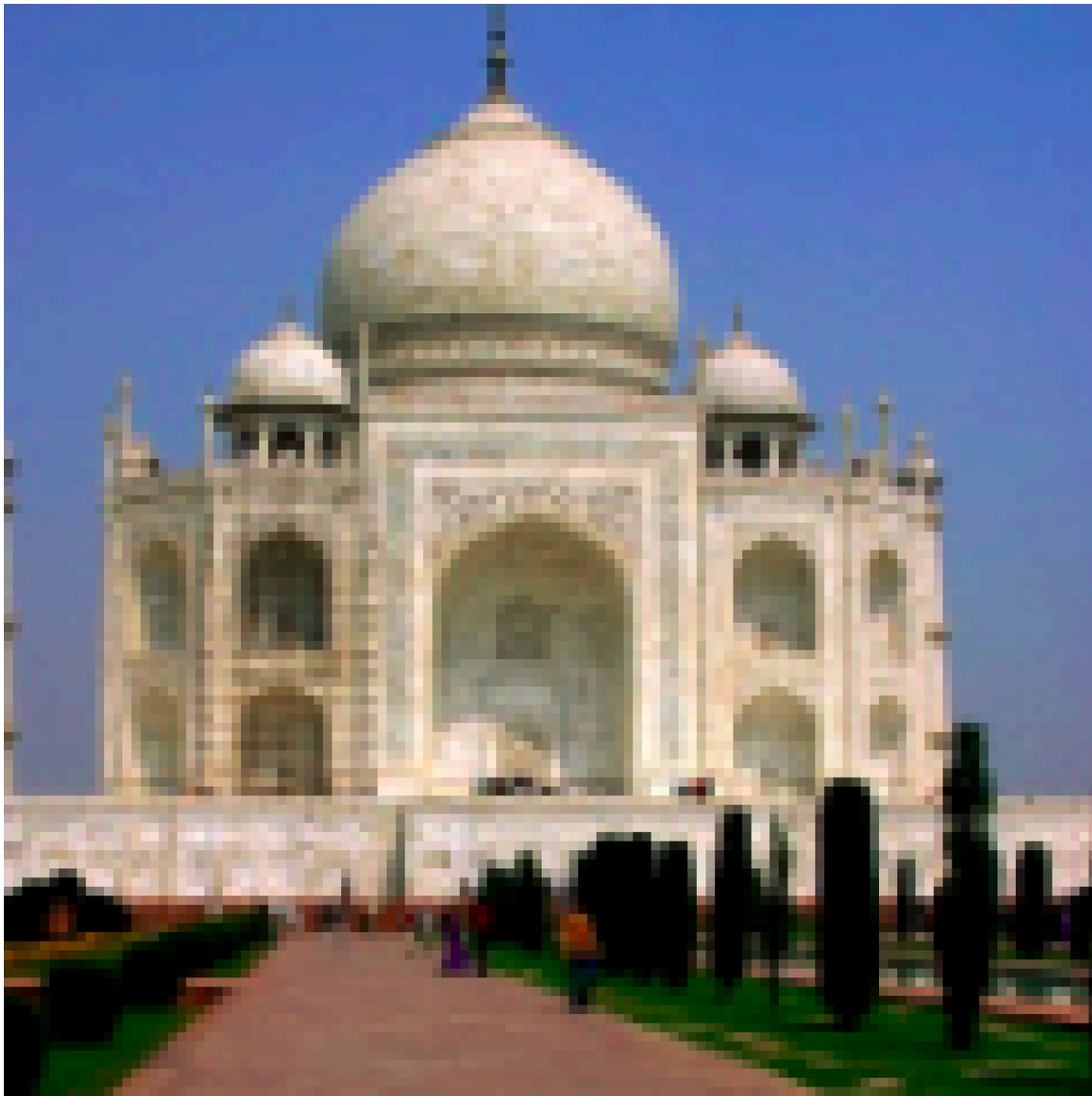
El ejemplo del que hemos hablado previamente (en otras clases) es el de la derecha. Al rasterizar una primitiva, los vértices que le dieron origen pueden tener valores asociados (por ejemplo color, normales, coord. de textura) y el rasterizador debe generarle a cada fragmento que produce sus propios valores interpolados.

EJEMPLOS



Además de valores obviamente escalares como colores (son 3 escalares), temperaturas o alturas, se pueden interpolar otras "cosas" más complejas como curvas y formas (veremos más adelante cómo se define una curva o silueta, y dependerá de ese "cómo" si la interpolación resulta simple o compleja, por ahora es solo un ejemplo sin mayores detalles).

EJEMPLOS



Cuando "agrandamos" una foto estamos queriendo ver "entremedio" de los pixeles que capturó la cámara. Cualquier "filtro" que intente disimular el pixelado está inventando filas y columnas de pixeles nuevas en medio de las preexistentes, y obviamente el color de cada nuevo pixel dependerá de los preexistentes más cercanos. Estamos interpolando colores en los pixeles nuevos.

EJEMPLOS



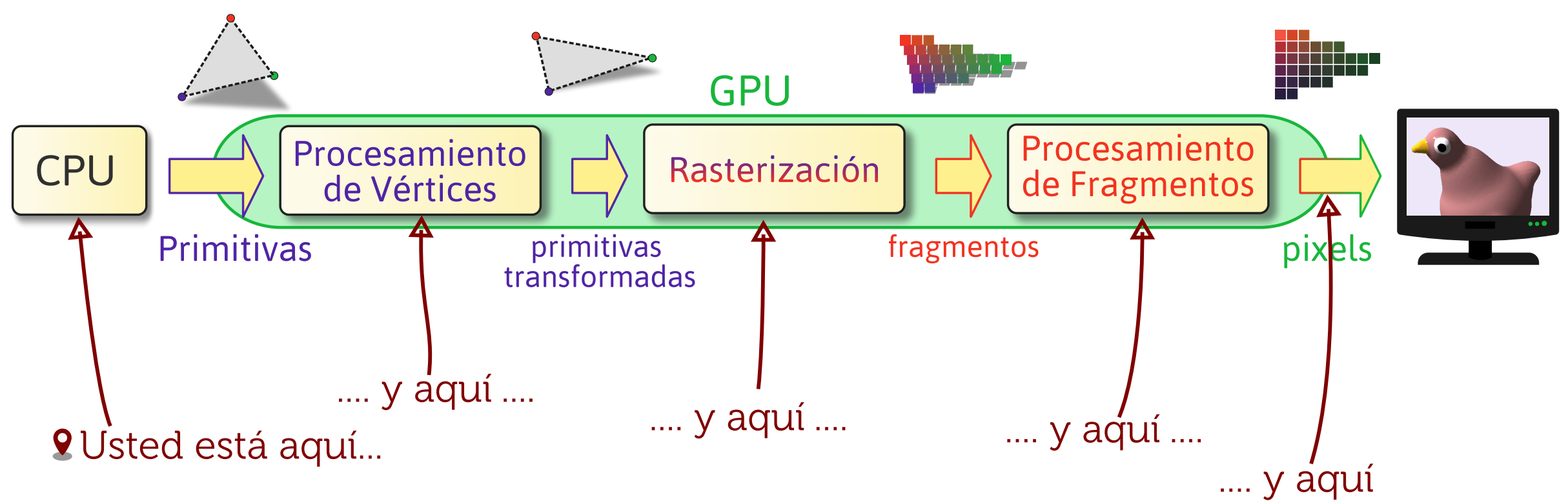
Un poco más interesante que el ejemplo anterior es interpolarle colores a píxeles que ya existían. Aquí queremos eliminar la reja, así que haremos de cuenta que esos píxeles "no existen" (son posiciones sin color asociado) y les interpolaremos un color nuevo a partir de los que sí (los que no están tapados por la reja).

EJEMPLOS



En este último ejemplo se interpola color y posición. Cuando mezclamos las dos caras, por ejemplo el color del ojo derecho en la mezcla debería ser una mezcla de los colores de los dos ojos derechos originales. Pero si superponemos las imágenes, veremos que las posiciones de los ojos no coinciden. Así que la posición también será una posición interpolada entre ambas para que en la transición de una cara a la otra varíe correctamente la forma y no sea un simple fading entre ambas imágenes.

USTED ESTÁ AQUÍ



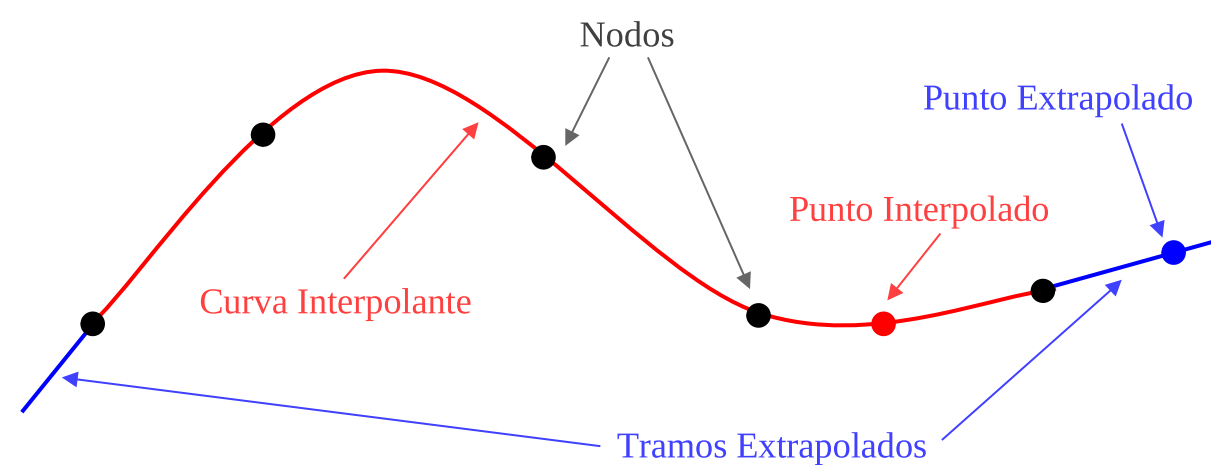
Interpolación se utiliza en mayor o menor medida por todos lados; no está asociada a una etapa del pipeline en particular. Algunos ejemplos en diferentes lugares del pipeline:

- La interpolación de alturas/terrenos, el morphing de imágenes y formas, en la mayoría de los casos la animación por fotogramas claves, etc, son ejemplos de uso por parte de la aplicación.
- Algunos trucos para producir efectos o animaciones directamente en la gpu utilizan interpolación dentro del vertex shader. Podría tener por ejemplo por cada vértice dos posiciones, inicial y final para una animación, y en cada frame generar un intermedia entre ambas según el tiempo (símil morphing).
- Ya vimos que la rasterización debe interpolar, para cada fragmento que genera, los valores que traían asociados los vértices (color, normal, profundidad, coordenadas de textura, etc).
- En el procesamiento de fragmentos también directa o indirectamente aparece la interpolación. Por ejemplo, si aplico una textura, cuando veo el objeto de cerca y hay que "agrandar" la imagen de la textura.
- Cuando el color resultante luego del procesamiento de fragmentos tiene un alpha/transparencia, la mezcla entre el color previo en ese pixel de la imagen, y el nuevo según el alpha también puede verse como una interpolación entre ambos colores.

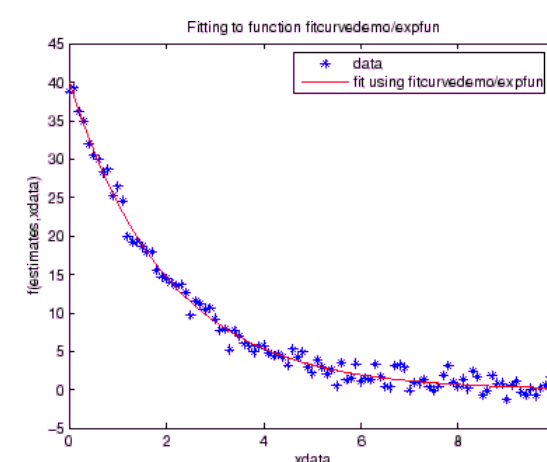
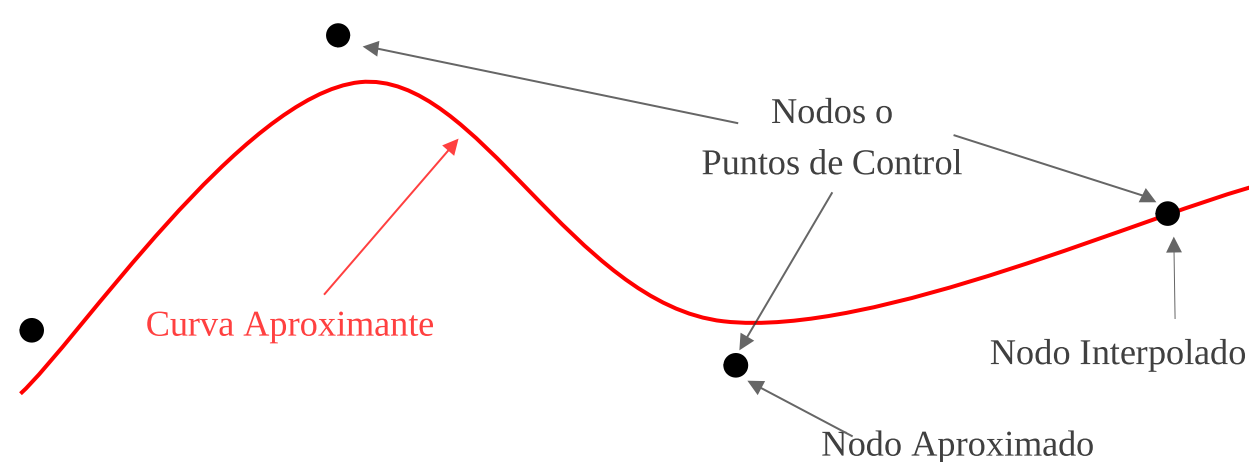
En algunos casos, como el blending, la rasterización, o las texturas, no tendremos que implementarla, sino que será parte de la funcionalidad de la GPU; pero sí será importante entender su funcionamiento para utilizar correctamente estas funcionalidades.

INTERPOLACIÓN VS. APROXIMACIÓN VS. EXTRAPOLACIÓN

• Interpolación vs. Extrapolación.



• Interpolación vs. Aproximación.



Interpolación vs Aproximación

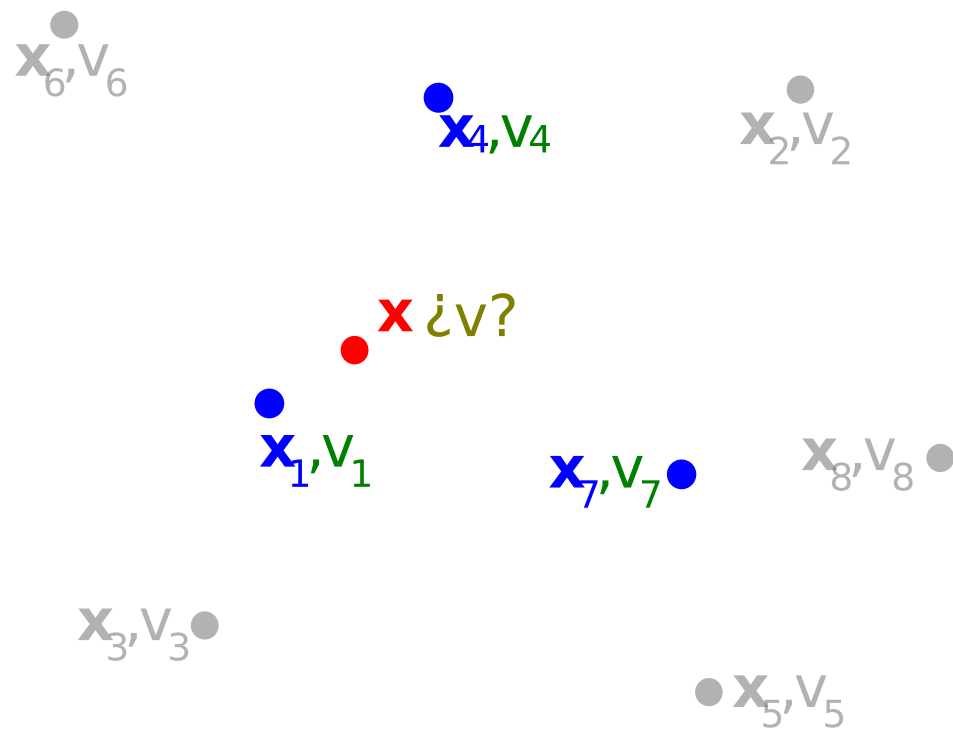
- Diremos que es interpolación cuando al calcular el valor para x coincidiendo en posición con uno de los puntos conocidos (x_i), el valor asociado calculado (v) siempre coincide con el valor asociado original de ese punto (v_i).
- Si no siempre coincide, diremos que estamos aproximando en lugar de interpolando.
- Esto es útil por ejemplo cuando los datos tienen ruido y no queremos que la función de interpolación "copie" todo ese ruido (regresión).

Interpolación vs Extrapolación

- Cuando calculemos para un x que esté "entre" de los puntos fijos/conocidos, diremos que estamos efectivamente interpolando.
- Si x está "fuera" de esos puntos estaremos extrapolando.
- Habría que definir mejor cuando definamos un método qué se entiende por "entre" o "fuera", pero en general se tratará de ver si el punto variable x cae dentro del *convex-hull* de los puntos fijos.

MÉTODOS LOCALES VS. GLOBALES

- ¿Cuáles nodos y valores conocidos utilizar?



- Todos -> Método Global
- Solo algunos -> Método Local

- Los métodos **globales** calculan un valor para el nuevo x utilizando todos los valores conocidos.
- Dado que en general se espera que el valor inventado asociado a x se parezca más a los valores asociados de los puntos más cercanos, muchos métodos considerarán para el cálculo solamente N puntos/valores cercanos y no todos; estos serán los métodos **locales**.

PROMEDIO PONDERADO

- Función de interpolación como promedio ponderado:

$$v(\mathbf{x}) = \sum_i \alpha(\mathbf{x}, \mathbf{x}_i) v_i = \sum_i \alpha^i(\mathbf{x}) v_i \quad \text{con} \quad \sum_i \alpha^i(\mathbf{x}) = 1$$

Normalmente:

$$\alpha^i \approx \frac{1}{|\mathbf{x}_i - \mathbf{x}|}$$

Métodos locales:

$$\mathbf{x}_i \text{ lejano} \Rightarrow \alpha^i = 0$$

- Lo que define a un método es:
 - ¿Cuáles nodos/valores elijo para calcular la interpolación?
 - ¿Cómo "peso" esos valores para obtener el nuevo?

La función de interpolación puede verse como un promedio ponderado de todos los valores conocidos. La forma de definir los pesos es lo que diferencia a un método de otro. Por ejemplo, en método "lineales", las funciones de peso serán funciones lineales; en métodos locales los pesos para valores asociados a nodos alejados serán 0.

Por lo tanto la pregunta importante es ¿cómo defino los pesos?, pero está implícito en ella ¿cuáles valores considero y cuáles no? (¿cuáles pesos serán 0?).

En todo método "razonable" los pesos asociados a valores más cercanos al punto x deberían ser mayores que los pesos asociados a valores lejanos.

INTERPOLACIÓN AFÍN

Una **combinación** afín es un promedio ponderado de puntos.

La **interpolación** utiliza para el nuevo valor asociado los mismos pesos que en la combinación afín nos darían el punto variable:

1) Encontrar pesos tales que:

$$\mathbf{x} = \sum_i \alpha^i \mathbf{x}_i \quad \text{con} \quad \sum_i \alpha^i = 1 \quad \Rightarrow \quad \boxed{\alpha^i = ?}$$

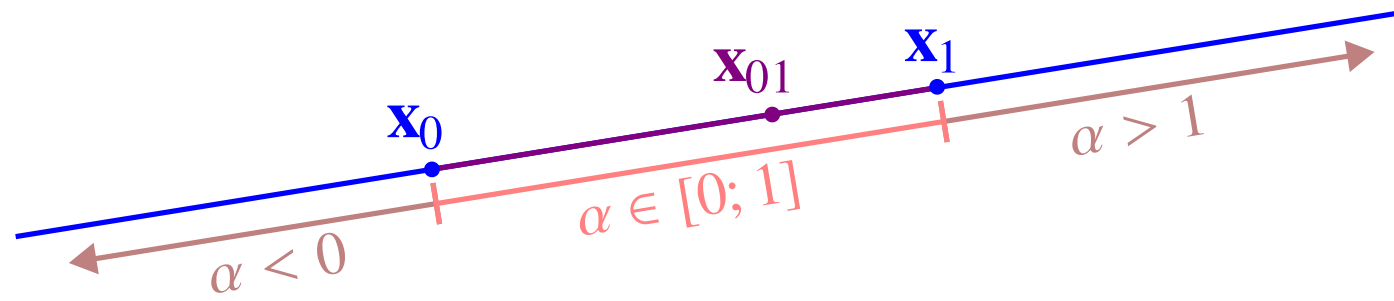
2) Usar esos pesos para calcular el nuevo valor:

$$v(\mathbf{x}) = \sum_i \alpha^i v_i$$

Notar que la combinación afín es una combinación lineal restringida a que los pesos sumen 1.

Si encontramos el conjunto de pesos que permiten obtener el punto variable a partir de los puntos fijos (todos estos puntos son dato, los fijos y el variable), podemos usar esos mismos pesos para calcular el valor asociado al punto variable (esta es la incógnita, el valor a "inventar") a partir de los valores asociados a los puntos conocidos.

COMBINACIÓN AFÍN



$$x_{01} = \alpha^0 x_0 + \alpha^1 x_1 \quad \text{con} \quad \alpha^0 + \alpha^1 = 1$$

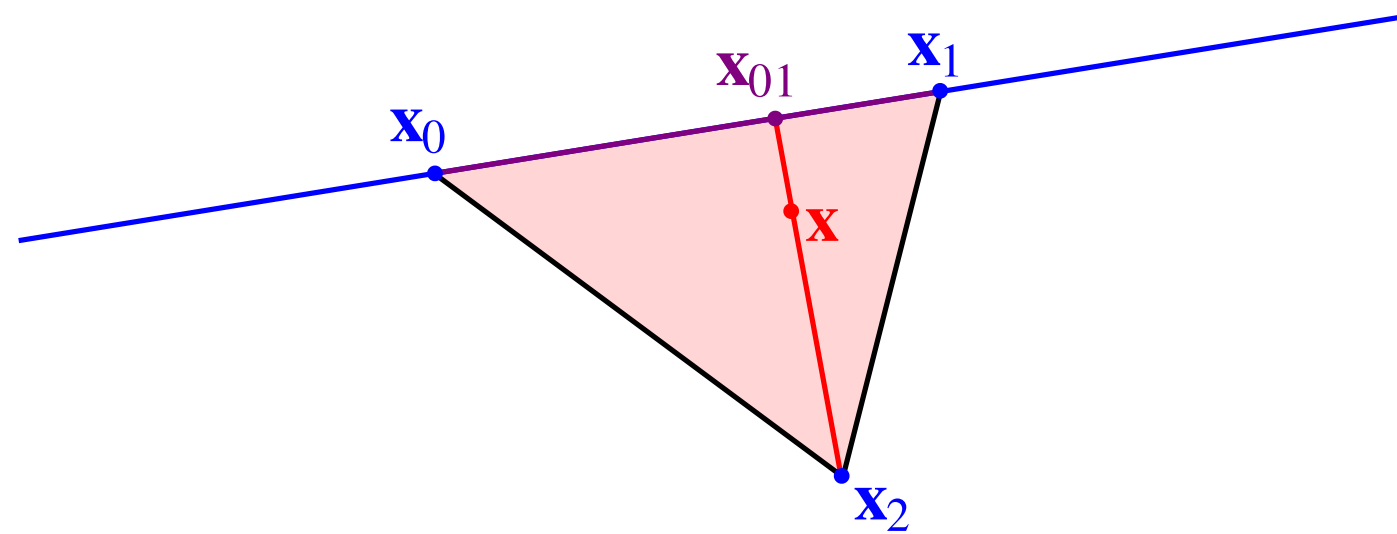
La "expansión afín" de 2 puntos (distintos) da la recta. Esto significa que mediante combinación afín de esos dos puntos puedo obtener cualquier otro de la recta que definen.

Anteriormente (repaso de álgebra) planteamos un punto de la recta como $p = p_0 + \alpha(p_1 - p_0)$.

- Aquí teníamos un solo α (que sería como α^1), y tiene sentido que la ecuación tenga una sola variable independiente si consideramos la restricción de que los pesos deben sumar 1 ($\alpha^0 = 1 - \alpha^1$).

- Si uso un α entre 0 y 1, obtengo un punto del segmento que los une (interpolación); mientras que si uso α s mayores a 1 o menores a 0 tendré puntos de la recta pero "fuera" del segmento (extrapolación).

COMBINACIÓN AFÍN



$$\mathbf{x}_{01} = \alpha^0 \mathbf{x}_0 + \alpha^1 \mathbf{x}_1 \quad \text{con} \quad \alpha^0 + \alpha^1 = 1$$

$$\mathbf{x} = \beta^{01} \mathbf{x}_{01} + \beta^2 \mathbf{x}_2 \quad \text{con} \quad \beta^{01} + \beta^2 = 1$$

$$\mathbf{x} = \beta^{01} (\alpha^0 \mathbf{x}_0 + \alpha^1 \mathbf{x}_1) + \beta^2 \mathbf{x}_2$$

$$\mathbf{x} = \frac{\beta^{01} \alpha^0}{\gamma^0} \mathbf{x}_0 + \frac{\beta^{01} \alpha^1}{\gamma^1} \mathbf{x}_1 + \frac{\beta^2}{\gamma^2} \mathbf{x}_2 \quad \text{con} \quad \gamma^0 + \gamma^1 + \gamma^2 = 1$$

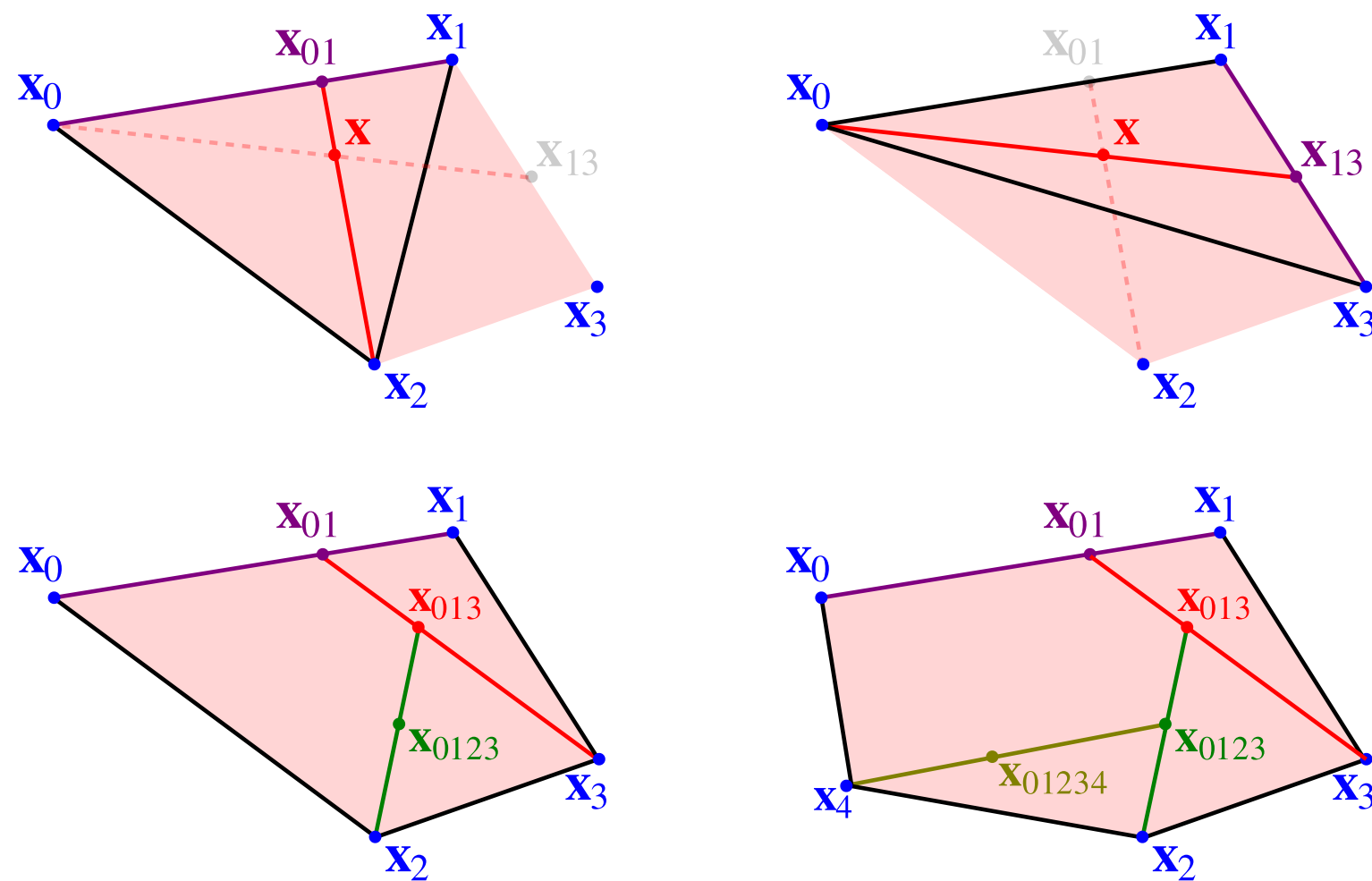
La expansión afín de 3 puntos (no colineales) da el plano.

Usamos la idea de partir de la combinación de 2 puntos de la slide anterior y combinar ese resultado con un tercero solo para que sean más fáciles de ver ciertas propiedades, como que afín de afín es afín, o que si los pesos (alphas y betas... o gammas) están entre 0 y 1 la combinación sigue siendo convexa y el punto x queda dentro del triángulo.

Es importante notar que si usáramos otra arista para el primer paso (por ejemplo, primero obtener un x_{12} sobre la arista que une x_1 con x_2 ; y luego a ese combinarlo con x_0) podríamos (con los pesos adecuados para alpha y beta) llegar al mismo punto final. Y en ese caso, habríamos usado alphas y betas diferentes, pero habríamos llegado a los mismos gamma. Los 3 pesos finales son únicos.

No será esta (usando un punto intermedio interpolado sobre una arista) la forma de obtener los pesos (finales, los gamma) en el caso del triángulo. Veremos en breve cómo calcularlos directamente.

COMBINACIÓN AFÍN



La expansión afín de 4 puntos no coplanares da el espacio 3D completo. Pero en este ejemplo ahora estamos agregando más puntos sobre el mismo plano.

Las dos figuras de arriba muestran que en N dimensiones, si tengo más de $N+1$ puntos, los pesos no son únicos. Aquí estamos en el plano (2 dimensiones), pero tenemos 4 puntos. A la izquierda llego a x sin usar x_3 , a la derecha al mismo x sin usar x_2 . Hemos perdido la "independencia afín", ya que de los cuatro puntos fijos, podemos tomar uno y plantearlo como combinación afín de los otros tres.

Las dos figuras de abajo muestran qué pasa si sigo agregando puntos. Puedo seguir planteando cada punto nuevo como combinación afín de los originales. Esta combinación no será única. Por ejemplo, se podría lograr pesando todos los puntos (como inducen a pensar estas dos últimas figuras), o solo 3 (como parecía en las dos de arriba). Lo interesante de la construcción "incremental" (ir agregando de a uno e interpolándolo con el resultado anterior) es que hace más evidente el hecho de que si hacemos combinaciones convexas, el resultado seguirá siendo convexo: estará dentro de lo que se denomina el "Convex Hull" (envoltorio convexo) del conjunto de puntos.

COORDENADAS BARICÉNTRICAS

Cuando los puntos fijos son afinmente independientes:

Pesos para la interpolación afín = "Coordenadas baricéntricas"

¿Cómo calcularlos dado un punto variable $\mathbf{x}$?

- Notar que:
 - Deben ser **funciones lineales**.
 - Deben valer **1 sobre un vértice y 0 en los demás**.
- Busquemos "algo" que cumpla con estos requisitos.

A los pesos de la interpolación afín, cuando partimos de un conjunto de puntos con independencia afín, también se los llama "coordenadas baricéntricas" o "coordenadas de área" (o en el contexto de la mecánica computacional y los métodos numéricos, "funciones de forma lineales").

- Lo de "coordenadas" tiene que ver con que son una forma de definir la posición de un punto, aunque ya no respecto a un sistema de referencias "normal" como cuando hablamos de x, y, z , sino respecto de $N+1$ puntos (en ND) arbitrarios.

La pregunta importante que queda por responder es ¿cómo calculo los pesos si conozco el punto?, que era el primer paso para la interpolación afín.

- Solemos llamar a esto la "inversión" de la interpolación, porque en lugar de tener pesos y usarlos para encontrar valores, tengo valores (en este caso posiciones) y quiero encontrar los pesos.

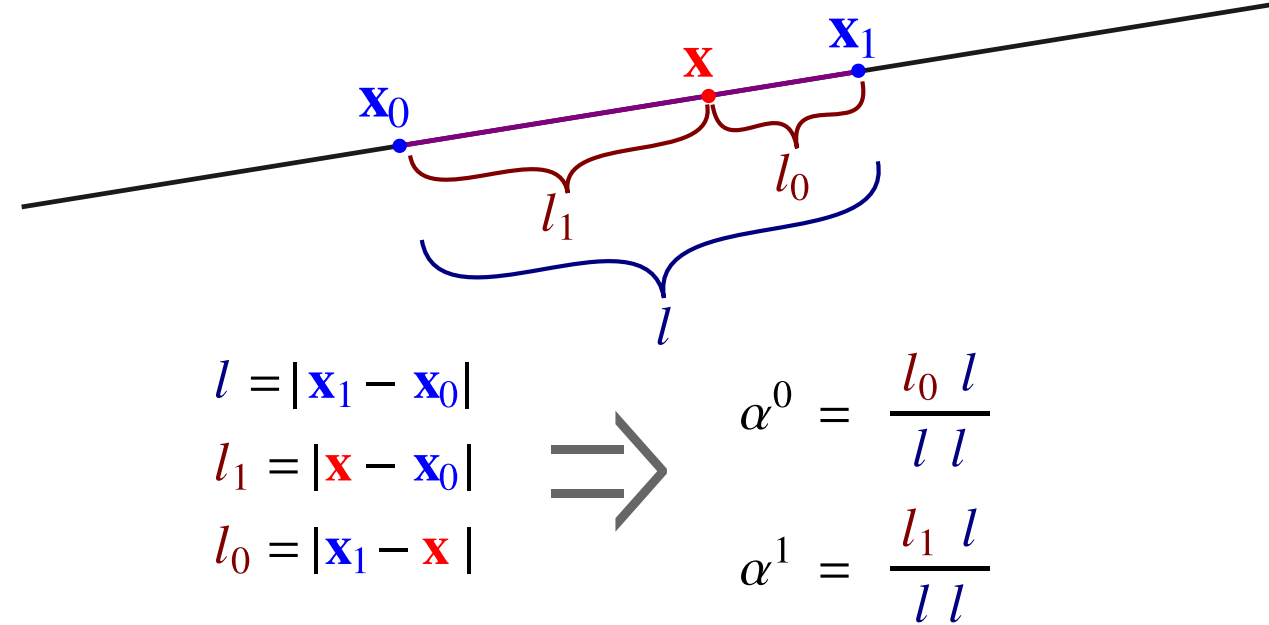
- Necesitamos entonces una función que sea lineal (N funciones en realidad, una por peso, pero todas tendrán la misma forma).

- Pero además sabemos cuánto debe valer esa función en ciertos puntos. Por ejemplo, si en un triángulo queremos que de verdad interpole (no solo aproxime), cuando el punto variable coincide con un vértice, el peso de ese vértice debe ser 1 y los demás 0.

A partir de estas dos observaciones, podemos intentar deducir ecuaciones para calcular los pesos.

COORDENADAS BARICÉNTRICAS

¿Cómo calcularlas para dos puntos fijos?



• La **distancia** del punto móvil a uno fijo:

- es una función lineal.
- vale 0 en dicho punto.
- vale L en el otro punto.

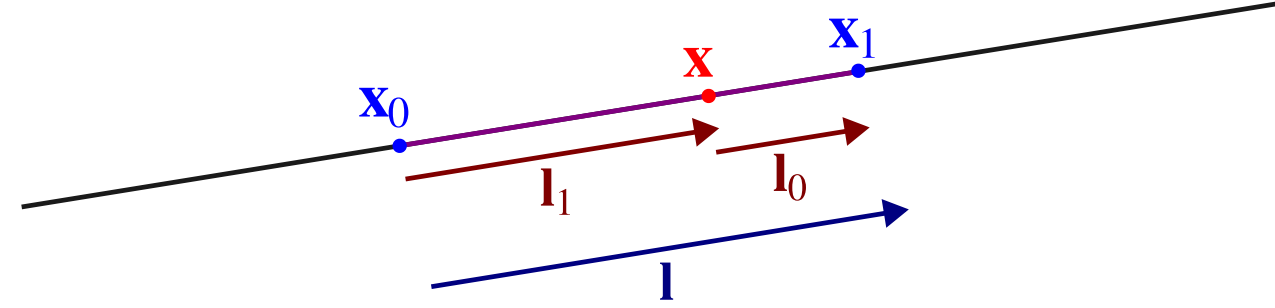
Cuando son 2 puntos, podemos usar las distancias desde dichos puntos al punto móvil. Cuando x coincide con uno de los vértices una es 0 y la otra es la longitud del segmento. Entonces, calculando la distancia del punto móvil x a un vértice, y normalizándola con la longitud total del segmento, obtenemos una función para el vértice del punto opuesto (por eso L_0 y L_1 están "cruzados" respecto a los puntos x_0 y x_1).

Por ejemplo cuando x coincide con x_0 , la distancia de x a x_0 es nula, y de x a x_1 es la longitud del segmento; y en este caso la interpolación debe dar x_0 , o sea que debe ser $x = 1 \cdot x_0 + 0 \cdot x_1$; por eso tenemos en este ejemplo tenemos la función para α^1 : $|x - x_0| / |x_1 - x_0|$. De igual forma se obtiene la de α^0 .

Sin embargo esto tiene todavía un problema grave: calculando la distancia de esta forma, los pesos nunca serán negativos (por los módulos). Entonces no sirve para extrapolar, o por ej, para determinar si el punto está o no dentro del segmento.

COORDENADAS BARICÉNTRICAS

¿Cómo calcularlas para dos puntos fijos?



$$\begin{aligned} \mathbf{l} &= \mathbf{x}_1 - \mathbf{x}_0 \\ \mathbf{l}_1 &= \mathbf{x} - \mathbf{x}_0 \\ \mathbf{l}_0 &= \mathbf{x}_1 - \mathbf{x} \end{aligned} \Rightarrow \begin{aligned} \alpha^0 &= \frac{\mathbf{l}_0 \cdot \mathbf{l}}{\mathbf{l} \cdot \mathbf{l}} \\ \alpha^1 &= \frac{\mathbf{l}_1 \cdot \mathbf{l}}{\mathbf{l} \cdot \mathbf{l}} \end{aligned}$$

- Para poder extrapolar (signo):
 - proyectar sobre una dirección "preferencial"
 - el vector $\mathbf{l}$ tiene la dirección adecuada

Para obtener un signo, en lugar de sacar el módulo, para convertir el vector $(x-x_0)$ en un escalar, lo multiplicaremos por otro vector de igual dirección, con el sentido adecuado para que el signo de ese producto escalar coincida con el que queremos para el peso.

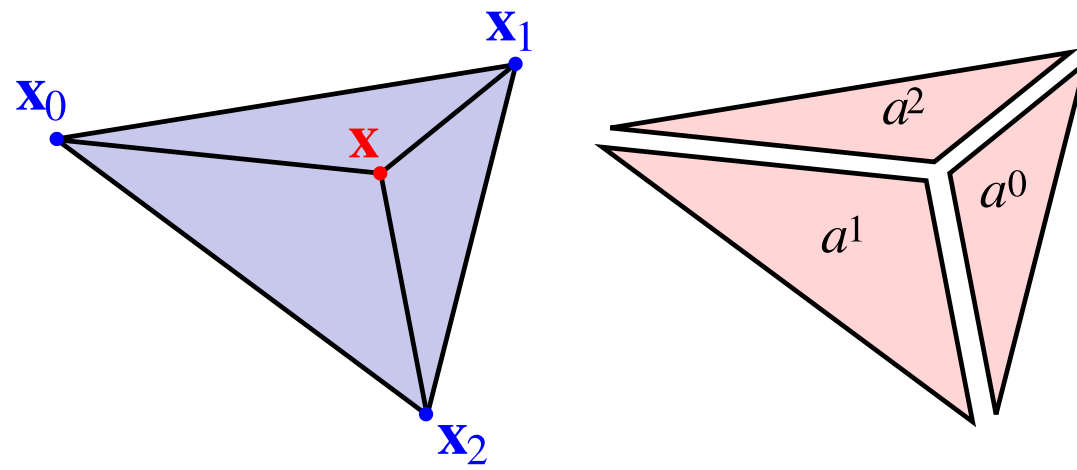
Recordar que el producto escalar se puede calcular como $|a| \cdot |b| \cdot \cos(\text{ang})$; si los vectores son paralelos, el coseno será +1 o -1, y dará el signo.

Para mantener la escala (porque además del signo, estamos escalando según el módulo de ese vector), haremos lo mismo con el denominador.

El vector que ya tenemos disponible y podemos usar para esto es justamente $\mathbf{l} = (\mathbf{x}_1 - \mathbf{x}_0)$.

COORDENADAS BARICÉNTRICAS

¿Cómo calcularlas para tres puntos fijos?



$$\mathbf{a} = (\mathbf{x}_2 - \mathbf{x}_0) \times (\mathbf{x}_1 - \mathbf{x}_0)$$
$$\mathbf{a}_2 = (\mathbf{x} - \mathbf{x}_0) \times (\mathbf{x}_1 - \mathbf{x}_0)$$
$$\alpha_2 = \frac{\mathbf{a}_2 \cdot \mathbf{a}}{\mathbf{a} \cdot \mathbf{a}}$$

- Usamos el **área** de los subtriángulos que forma **x**.
- Para obtener signo, nuevamente recurrimos a proyectar (sobre el vector área total).

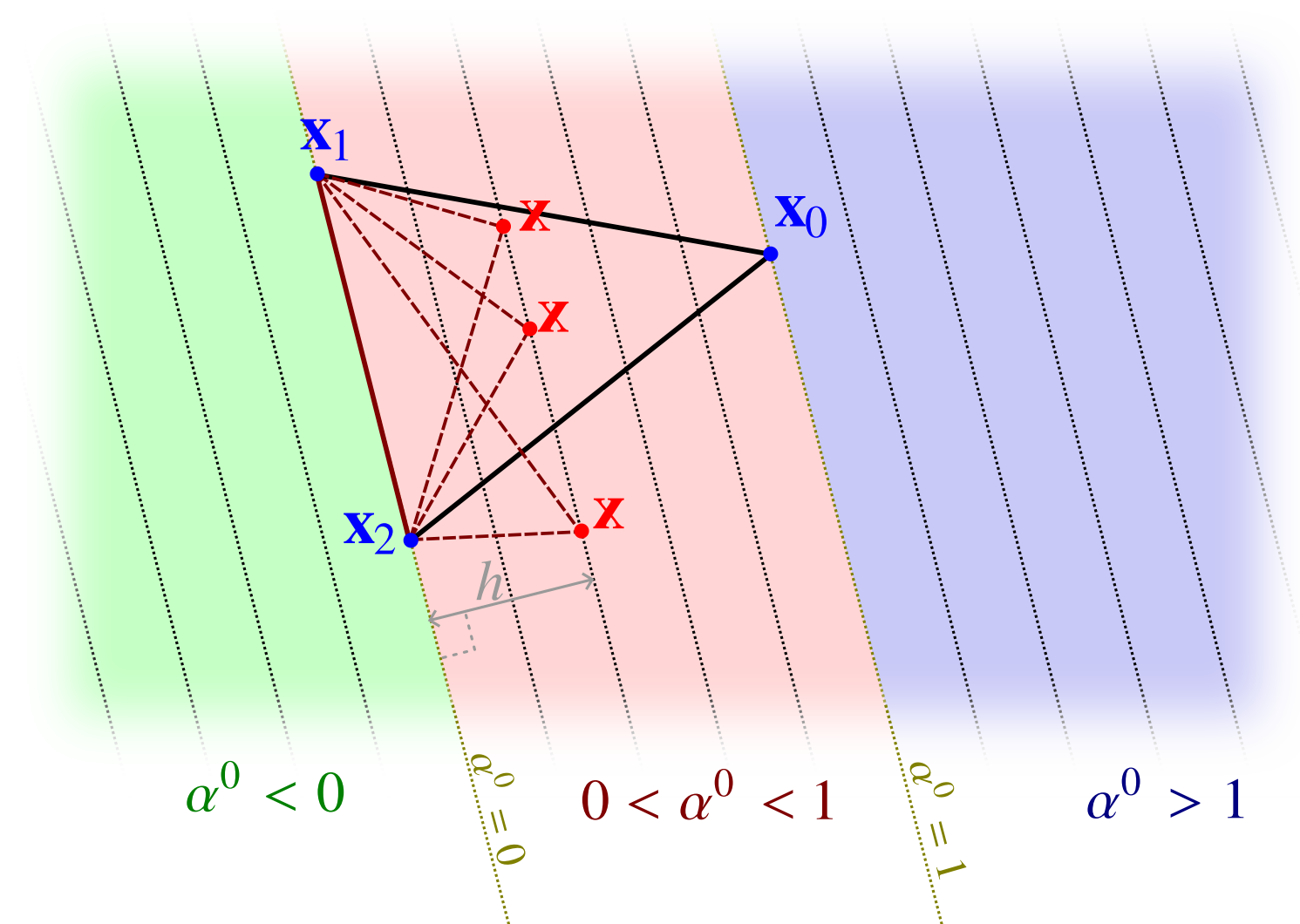
El área del subtriángulo que forma un lado del triángulo fijo con el punto móvil sirve de base para la función del alpha, de la misma forma que en el caso anterior usamos la distancia/longitud.

Supongamos que tomamos el sub-triángulo dado por x_0, x_1, x . Cuando x esté sobre el lado que va de x_0 a x_1 , esa área se hace 0. Si tomamos ese lado como base, a medida que nos "alejamos", el área (base*altura/2) crece linealmente: la base es fija (x_0, x_1), la altura varía linealmente si el pto x se mueve en dirección perpendicular. Cuando x llegue a x_2 , el subtriángulo será igual al triángulo fijo, por lo que las áreas serán iguales.

Para calcular el área usaremos el producto cruz: el módulo daba el doble del área que queremos, pero no concideramos el /2 porque iría tanto en el numerador como en el denominador; y no aplicamos módulo para no perder el signo y poder extrapolar (por eso hablamos de "vector" área, cuando el área en realidad es un escalar no negativo).

Nuevamente, si armamos los vectores y productos teniendo cuidado con el orden (si el punto está adentro del triángulo, todos los productos vectoriales deben dar para el mismo lado), entonces podemos proyectar los vectores de las subáreas sobre el vector del área total para obtener el signo (siempre serán paralelos, pues el producto vectorial da un resultado perpendicular, según la "regla de la mano derecha").

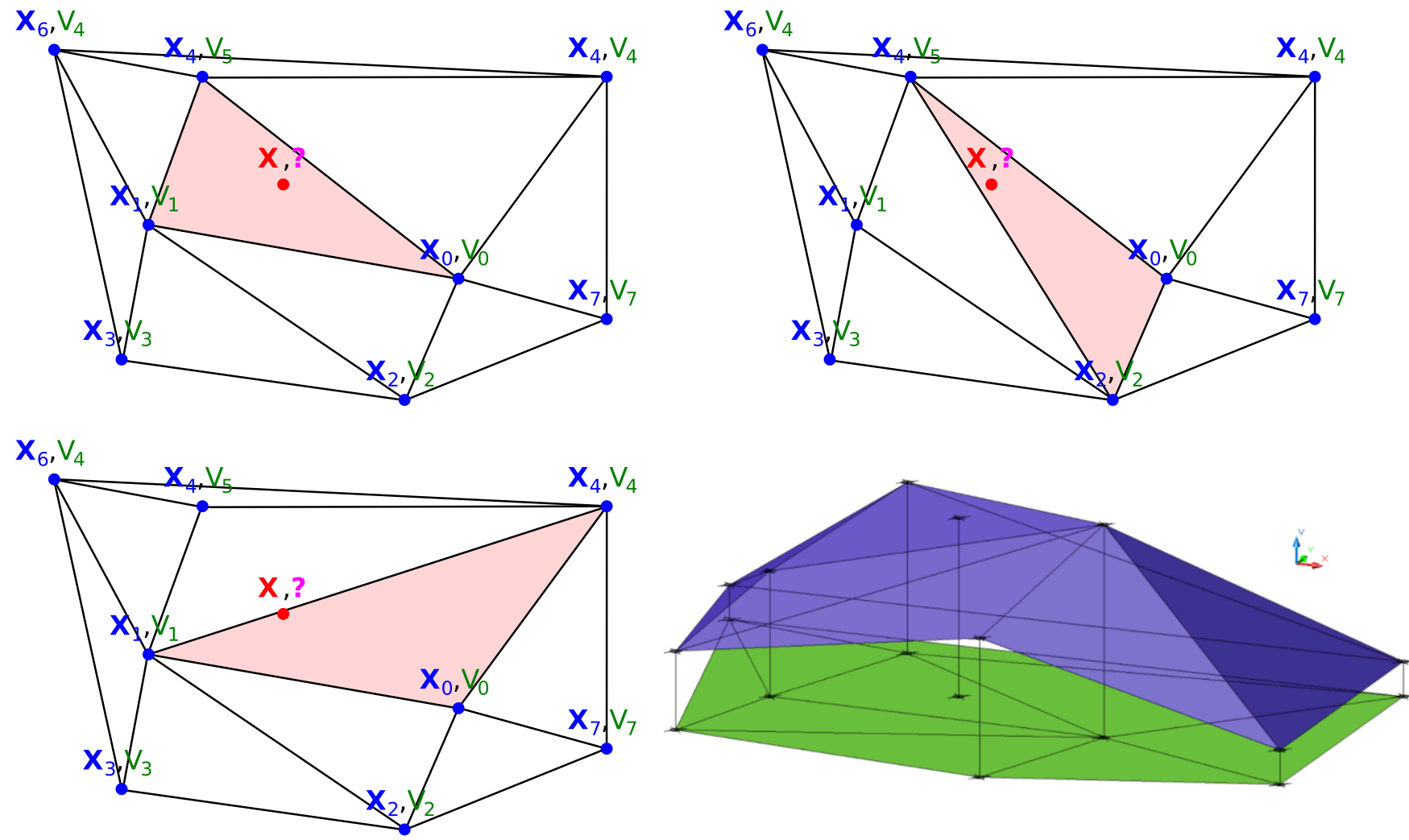
COORDENADAS BARICÉNTRICAS



Aquí se quiere representar el valor de α_0 de acuerdo a la posición de x . α_0 está asociado a x_0 , por lo tanto será el área del subtriángulo opuesto (el que tiene a x_1-x_2 como uno de sus lados). Todas las líneas paralelas a x_1-x_2 serán "isolineas" (es decir, líneas de α_0 constante). En particular, la que pasa por x_1 y x_2 es la isolinea de $\alpha_0=0$, y la que pasa por x_0 la de $\alpha_0=1$. Entre medio de ellas (zona roja), α_0 varía entre 0 y 1, a la izquierda (zona verde) se torna negativo, a la derecha (zona azul) será mayor a 1. Dado que varía linealmente, podemos tomar la altura del triángulo original (x_0, x_1, x_2) como referencia para estimar cualquier valor de α_0 según la distancia (de x) perpendicular a la recta de x_1-x_2 .

El mismo análisis se puede hacer para las otras 2 coordenadas. Veremos que solo dentro del triángulo los tres alphas son positivos, y nunca los 3 negativos. En caso de haber uno o dos negativos, no solo indicarán que el punto está fuera, sino también aproximadamente en qué dirección.

INTERPOLACIÓN AFÍN CON "MUCHOS" PUNTOS



Armar triangulación y usar solo el triángulo que contiene a **x**

Si se tienen muchos puntos conocidos y se quiere utilizar igualmente interpolación afín, deberemos seleccionar para cada x variable solo 3 puntos fijos.

- Deberían ser 3 puntos fijos que "contengan" al punto variable, y seleccionados de tal forma que garanticen algún grado de continuidad en la variable interpolada.

- En el caso del plano (que es el más habitual) utilizaremos un "triangulación". Esto es, partición del convex-hull en triángulos. Lo de "partición" implica que los triángulos cubren todo el convex-hull, y que no se solapan entre sí.

- Sobre una recta o curva (1D), la partición es trivial. Pero en el plano (2D) o espacio (3D) no lo es. Para un conjunto de puntos dados, puede haber muchas triangulaciones (o en 3D, tetraedrizaciones) posibles.

- La "calidad" de la solución dependerá en cierta medida de la "calidad" de la triangulación. Preferiremos triángulos más parecidos a un equilátero, con lados y ángulos parejos.

- No entraremos en detalle por ahora, pero en general, triángulos aplastados serán malos para la interpolación.

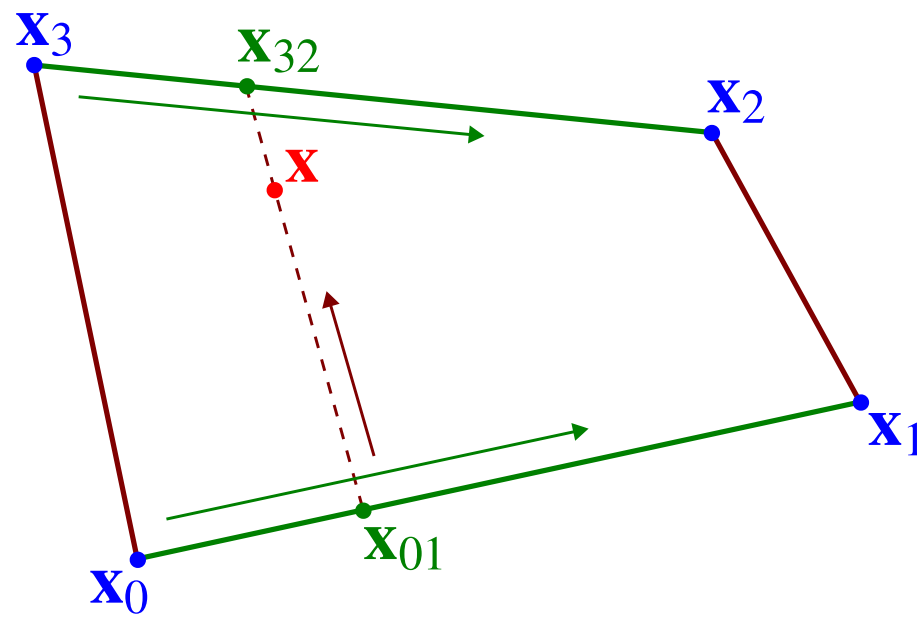
- En la unidad de ordenamiento espacial describiremos una triangulación en particular (Delaunay) que resulta ideal por sus propiedades para esta aplicación.

Una vez definida una triangulación, resta plantear alguna forma de encontrar eficientemente cuál es el triángulo que contiene a un x variable dado.

- Si tomamos un triángulo al azar, y usamos los propios pesos de la interpolación afín para saber si contiene a x . En caso de no contenerlo, el peso negativo nos indicará aproximadamente en qué dirección debemos movernos por la triangulación para acercarnos a x (a esto se lo conoce como "búsqueda lineal").

- En la unidad de ordenamiento espacial veremos más detalles, y cómo partir de algo mejor que un triángulo "al azar".

INTERPOLACIÓN BILINEAL



$$X_{01} = (1 - u)X_0 + uX_1$$

$$X_{32} = (1 - u)X_3 + uX_2$$

$$X = (1 - v)X_{01} + vX_{32}$$

$$X = (1 - v)((1 - u)X_0 + uX_1) + v((1 - u)X_3 + uX_2)$$

$$X = \underbrace{(1 - v)(1 - u)}_{\alpha^0} X_0 + \underbrace{(1 - v)u}_{\alpha^1} X_1 + \underbrace{v(1 - u)}_{\alpha^3} X_3 + \underbrace{vu}_{\alpha^2} X_2$$

$$\alpha^0 + \alpha^1 + \alpha^2 + \alpha^3 = 1$$

Empezamos a ver ahora alternativas a la interpolación afín y lineal. Cuando tengamos 4 puntos, hay una alternativa en muchos casos simple que es la interpolación bilineal.

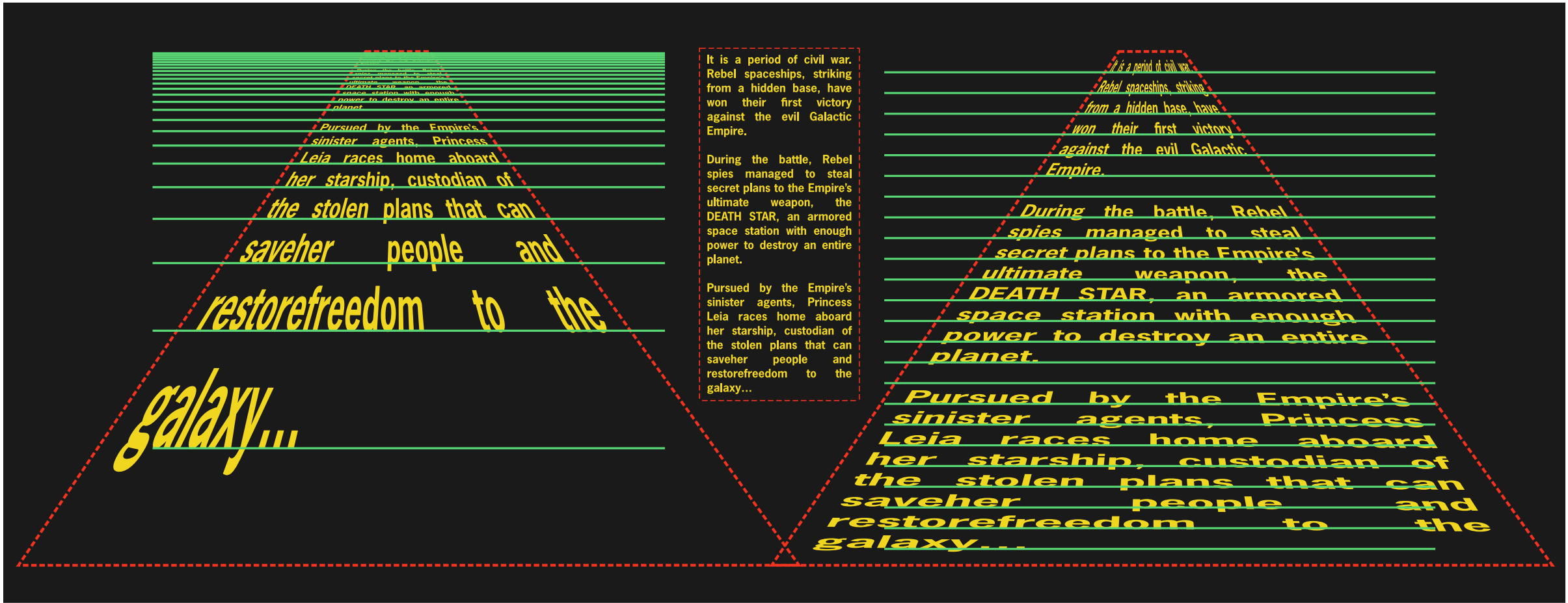
- Podemos obtener el punto x , haciendo primero dos interpolaciones afines con un mismo parámetro (u) en lados opuestos del cuadrilátero (x_{32} y x_{01} en el ejemplo); y luego otra interpolación afín con otro parámetro (v) entre ambos resultados.

- Se puede demostrar fácilmente que el resultado es una combinación afín (pero no lineal, hay productos $u \cdot v$) de los 4 puntos, y que los 4 pesos suman 1.

- Pero la ecuación no se plantea en función de 4 pesos, sino de solo 2 parámetros (u y v).

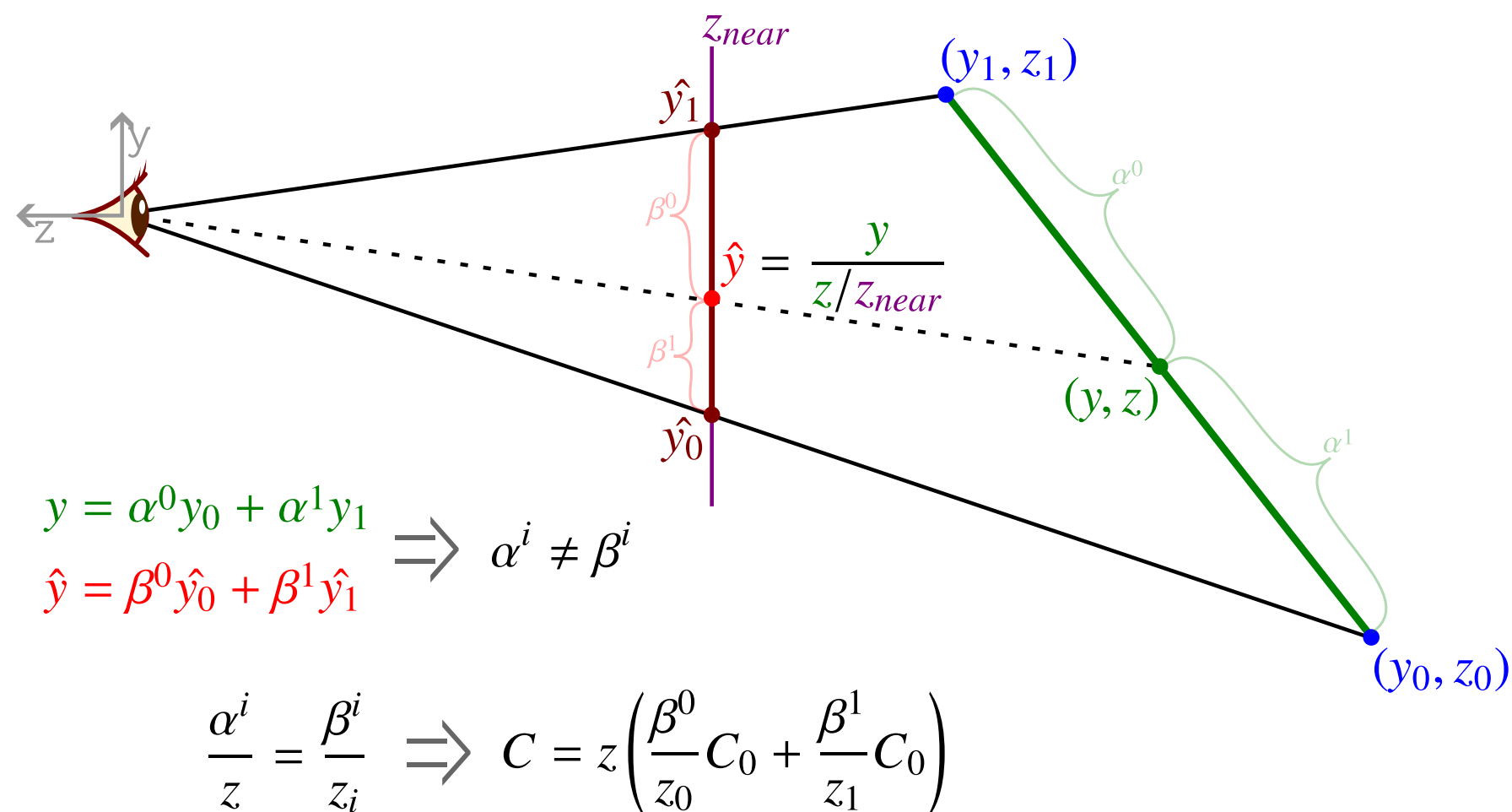
- Encontrar u y v en un caso general puede ser muy complicado (debido a la no-linealidad de las ecuaciones). Pero si el cuadrilátero es en realidad un cuadrado o rectángulo de lados alineados con los ejes, resulta muy simple; y es un caso que usaremos mucho (por ej, más adelante para texturas).

INTERPOLACIÓN HIPERBÓLICA



Otro caso donde la interpolación aún no es lo ideal es justamente en la rasterización. Claramente la versión correcta del texto alejándose en 3D es la de la derecha, donde las letras que están cerca "se ven" más altas que las que están lejos. Pero entonces la coordenada y en el texto (pensemos en el nro de renglón) no varía linealmente a lo largo del cuadrilátero (con la coordenada y de la imagen resultante). Sí lo hace en la imagen de la derecha, donde en el medio de la imagen está el renglón del medio del texto. Pero en la correcta, el centro de la imagen no coincide con el centro del texto.

INTERPOLACIÓN HIPERBÓLICA



⚠ Los pesos originales no coinciden con los pesos "proyectados"

En esta figura, el plano $z=z_{near}$ sería el plano de la imagen. El segmento rojo una primitiva proyectada, el verde la primitiva original. La línea de puntos muestra dónde se proyecta el centro de la primitiva original en la imagen, y no es en el centro. Ambas versiones de ese punto se pueden obtener por interpolación afín, pero los pesos no son los mismos: son 0.5 y 0.5 en la original (verde) y aproximadamente 0.6 y 0.4 en la versión proyectada (roja).

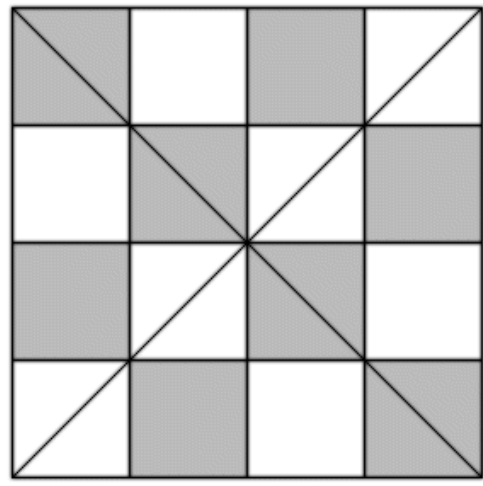
La interpolación hiperbólica tiene en cuenta esta diferencia. La idea es "corregir" los pesos utilizando información adicional. En el caso de la rasterización y la "corrección perspectiva", esa información adicional es la coordenada z , que se "pierde" al proyectar.

No exigiremos recordar estas ecuaciones en las evaluaciones (igualmente se pueden deducir fácilmente a partir del dibujo), pero sí es importante entender el problema y cuándo aplicar esta interpolación.

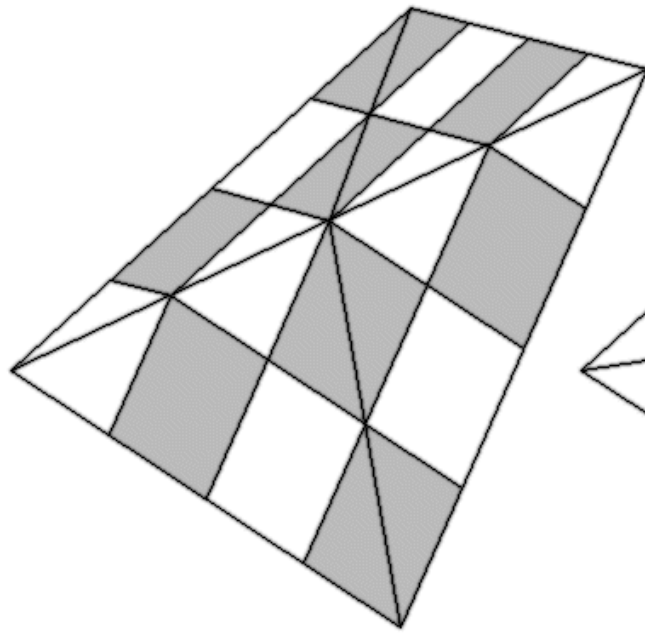
En la etapa de rasterización, la coordenada z del fragmento sí varía linealmente. Todo lo demás debería corregirse utilizando los z s, del fragmento (interpolado) y de los vértices (los originales antes de proyectar).

En general, los libros plantean la interpolación hiperbólica en función de un peso adicional w , que se relaciona con la 4ta coordenada del espacio proyectivo (lo que se conoce como coordenadas "homogéneas"). Veremos qué significa todo esto último en la unidad de transformaciones. Por ahora podemos pensarlo en función de z , la idea es exactamente la misma.

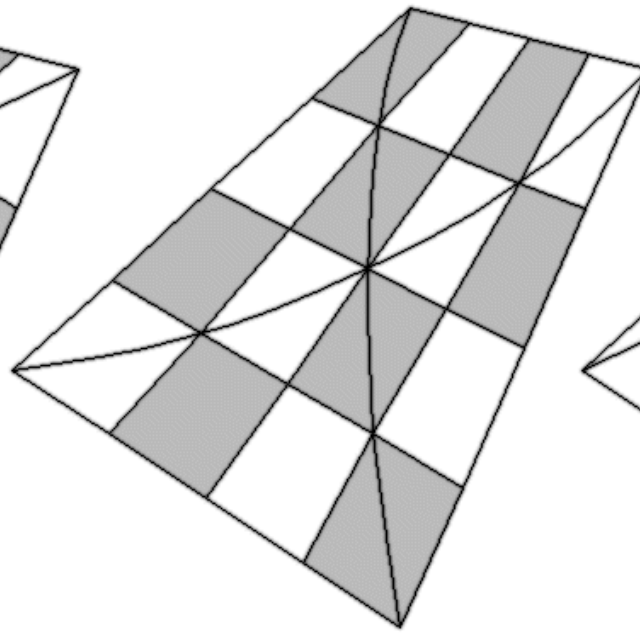
INTERPOLACIÓN EN CUADRILÁTEROS



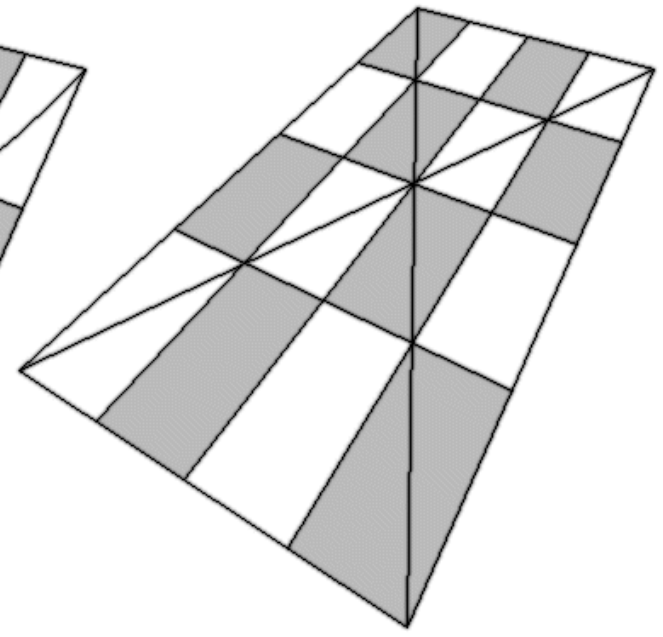
Original



Dos Lineales



Bilineal

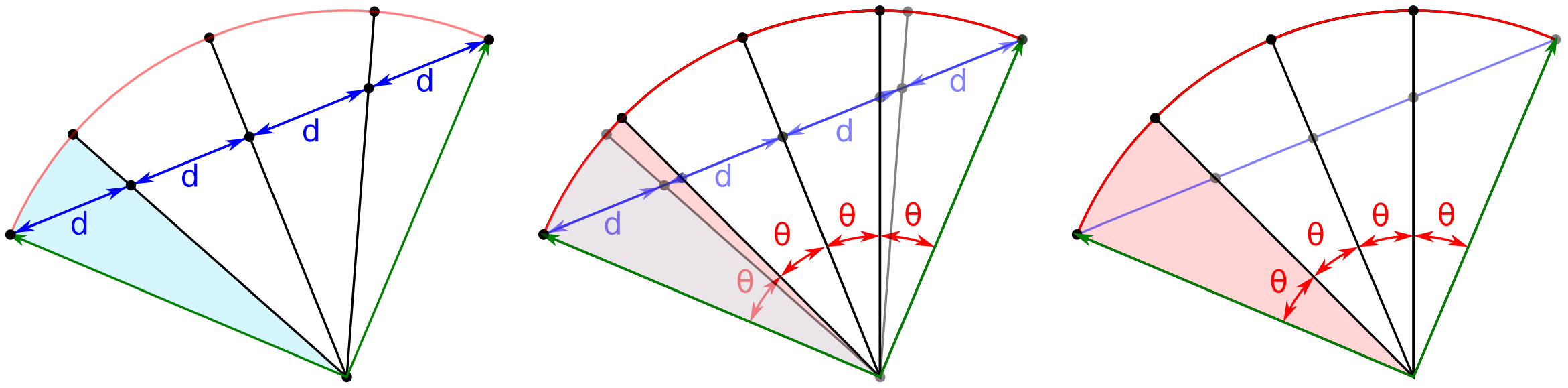


Hiperbólica

Aquí se compara el resultado de interpolar sobre un cuadrilátero de tres formas diferentes:

- Partiéndolo en dos triángulos y utilizando interpolación lineal en cada uno. En este caso será muy evidente que se partió en 2 triángulos porque habrá un discontinuidad (de la 1ra derivada, no del valor) sobre esa diagonal.
- El segundo aplica la bilineal. Se nota la no-linealidad al marcar las versiones interpoladas de los puntos que originalmente estaban en la diagonal, ya que así la nueva "diagonal" no es recta.
- El tercero es la hiperbólica. Las diagonales son rectas, pero no se cruzan en el centro de la versión proyectada.

INTERPOLACIÓN ESFÉRICA (SLERP)



- No es lo mismo
 - interpolar linealmente las "componentes" del vector (izquierda)
 - que interpolar linealmente el "ángulo" (derecha)

Cuando se trata de interpolar normales/direcciones, la interpolación afín no es correcta. Si interpolamos las componentes de dos vectores normales como lo hemos venido haciendo con las coordenadas de los puntos, el resultado será el mismo: si imaginamos a los vectores sobre un origen común, el vector interpolado apuntará a un punto en el segmento que une los extremos originales.

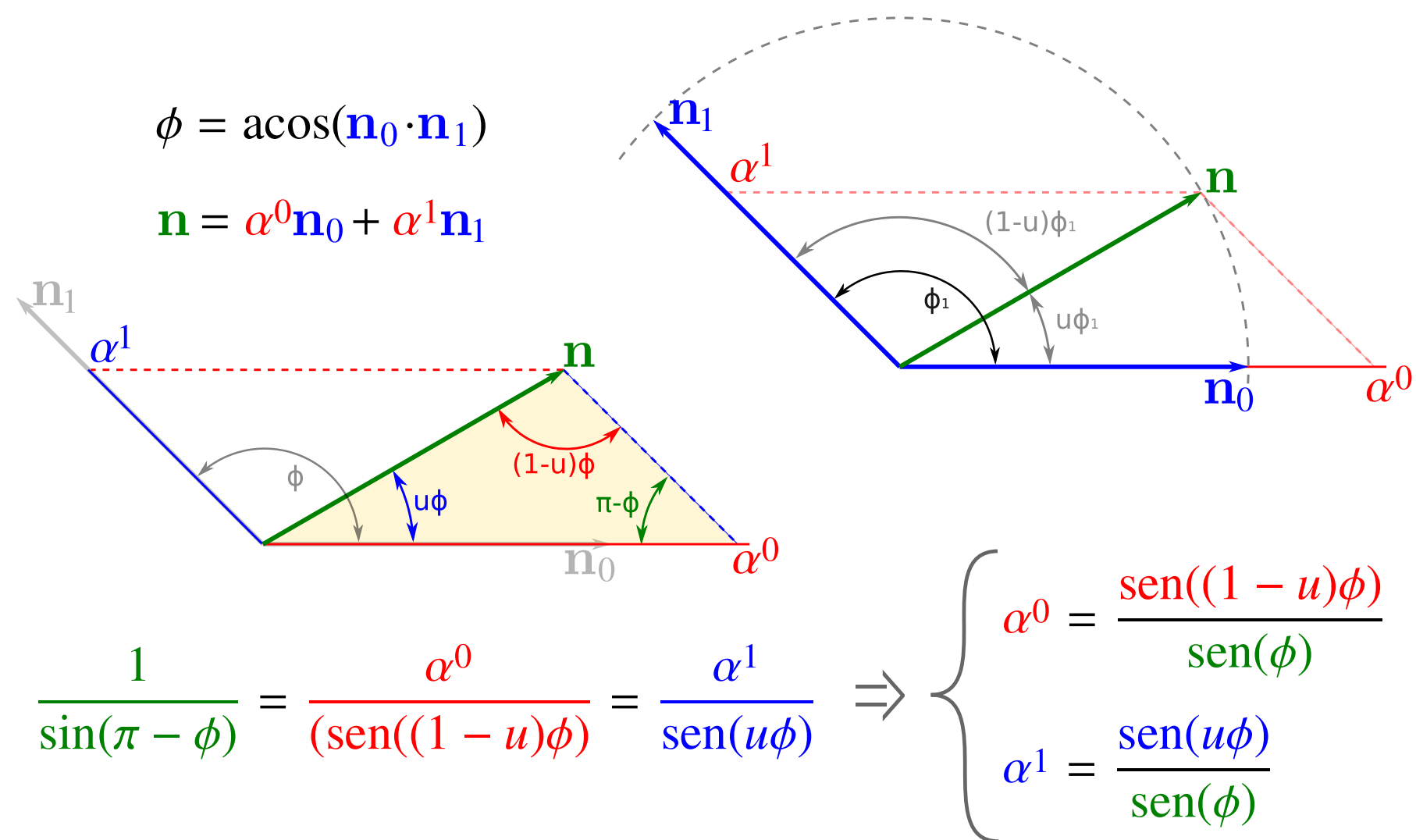
- Por un lado, este nuevo vector no será unitario. Esto no es un problema grave, dado que podríamos renormalizarlo.

- Por otro lado, no hay una relación lineal entre el ángulo de los vectores y la distancia sobre ese segmento. Es decir, que un punto a 1/4 del inicio en el segmento no se corresponde con un vector que forme con el 1ro 1/4 del ángulo original.

Si queremos dividir correctamente el ángulo, debemos usar otro tipo de interpolación: SLERP (esférica-lineal).

Notar que si bien en general no coincide la dirección del vector resultante si se interpola sobre el segmento (afín) o el ángulo (slerp), sí coincide justo para el centro ($\alpha=1/2$).

INTERPOLACIÓN ESFÉRICA (SLERP)



! slerp(slerp) no es slerp

Dadas las dos normales iniciales ($\mathbf{n}_1$ y $\mathbf{n}_2$), se pueden obtener las componentes de la interpolada esféricamente (donde el parámetro u recorre linealmente el ángulo ϕ) mediante una combinación lineal (no afín, no suman 1) de las originales.

Se pueden deducir las ecuaciones aplicando algunas relaciones trigonométricas (la suma de ángulos, el teorema del seno). No exigiremos estas ecuaciones en las evaluaciones, pero sí es importante entender la diferencia con una "simple" interpolación afín.

Sin embargo se debe notar que hay problemas al querer "extrapolar" este mecanismo a 3 direcciones. A diferencia de la interpolación afín, donde para el triángulo podíamos pensar que partíamos obteniendo un punto sobre un lado y luego interpolando con el 3ro, aquí eso no aplica. En la afín, los pesos resultantes siguen sumando 1 (afín de afín ES afín) y da lo mismo de qué lado se parte (si se llega al mismo punto, los pesos finales son únicos). En slerp, esto no ocurre (decimos que slerp de slerp NO es slerp), y entonces, por ejemplo, no da lo mismo el orden si hacemos slerp entre dos direcciones y luego el resultado con una tercera.

Usaremos slerp muy poco (solo cuando se trate de dos normales/direcciones). En el caso de la rasterización, todo se resuelve entre afín e hiperbólica, aún las normales dentro de un triángulo (por eso el fragment-shader debe re-normalizarlas antes de usarlas para calcular la iluminación).

CUATERNIONES

- Cuaternión: $q = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$
 - Componentes: $\{ a, b, c, d \}$
 - Base: $\{ 1, \mathbf{i}, \mathbf{j}, \mathbf{k} \}$
 - $\mathbf{i}.\mathbf{j} = \mathbf{k}, \mathbf{j}.\mathbf{k} = \mathbf{i}, \mathbf{k}.\mathbf{i} = \mathbf{j}$
 - $\mathbf{j}.\mathbf{i} = -\mathbf{k}, \mathbf{k}.\mathbf{j} = -\mathbf{i}, \mathbf{i}.\mathbf{k} = -\mathbf{j}$
- Representación escalar-vector:
 - $q = \langle a, \mathbf{u} \rangle; \mathbf{u} = \{b, c, d\}$
 - Vector: $\mathbf{u} = \langle 0, \mathbf{u} \rangle$
 - Escalar: $s = \langle s, \mathbf{0} \rangle$

Los cuaterniones son algo así como un número complejo que tiene tres partes imaginarias en lugar de solo una.

En un complejo $a+b\mathbf{i}$:

- a es la componente real y b la imaginaria;
- 1 e $\mathbf{i}$ son las "bases",
- teniendo $\mathbf{i}$ la propiedad de que multiplicada por sí misma da -1 .

En un cuaternión $a+b\mathbf{i}+c\mathbf{j}+d\mathbf{k}$:

- tenemos una componente real a y tres "imaginarias" b, c, d
- las bases son $1, \mathbf{i}, \mathbf{j}, \mathbf{k}$,
- $\mathbf{i}, \mathbf{j}$, o $\mathbf{k}$ al cuadrado dan -1 .
- y los productos entre $\mathbf{i}, \mathbf{j}$, o $\mathbf{k}$ funcionan como el producto vectorial si vemos a $\mathbf{i}, \mathbf{j}$, o $\mathbf{k}$ como los 3 vectores de una base ortogonal.

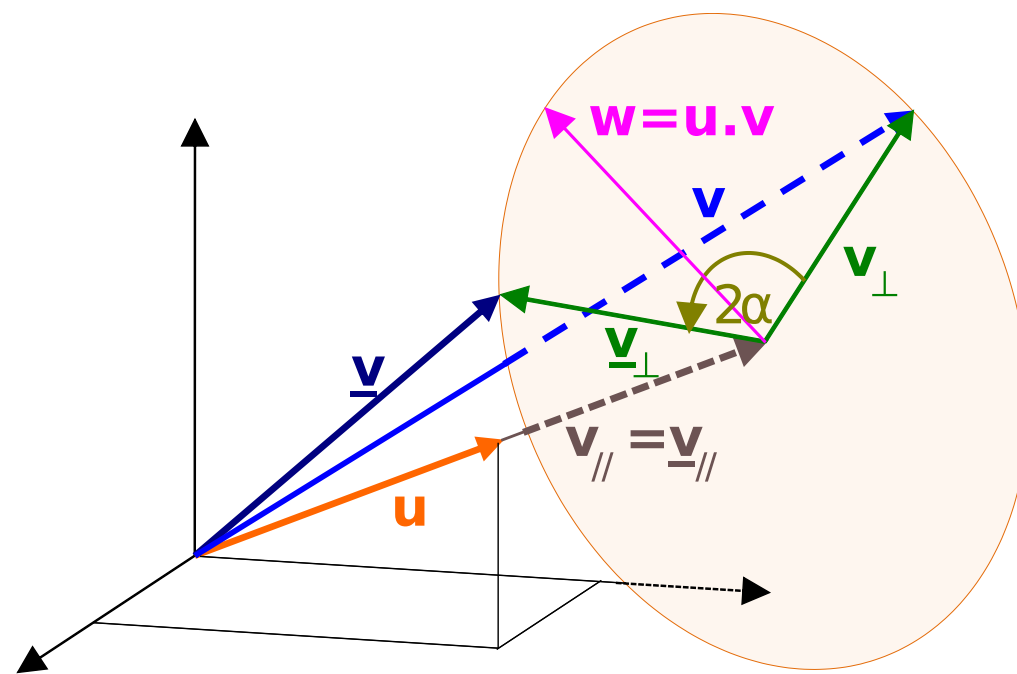
Por lo dicho, podemos pensar que las 3 componentes "imaginarias" son las componentes de un vector: el cuaternión está compuesto por un escalar y un vector 3D.

Hay además definiciones de:

- módulo del cuaternión: $(\sqrt{a^2 + \mathbf{u}^2})$, similar al módulo de un vector
- cuaternión conjugado: $q^* = \langle a, -\mathbf{u} \rangle$, similar al de un complejo
- cuaternión unitario: $q/|q|$, similar al vector "unitario"
- cuaternión inverso: $q^*(q^{-1}) = 1, q^{-1} = q^*/(|q|^2)$
- producto: tomar las 4 componentes, hacer todos contra todos, y aplicar las propiedades de los productos de las bases

ROTACIÓN CON CUATERNIONES

La transformación
 $\underline{\mathbf{v}} = \mathbf{q} \mathbf{v} \mathbf{q}^{-1}$
produce una rotación de un ángulo 2α alrededor de $\mathbf{u}$.



- Se pueden componer e interpolar "fácilmente".

La mayoría de los motores 3D modernos utilizan cuaterniones para representar rotaciones arbitrarias. La componente real codifica el ángulo, la parte vectorial codifica el eje sobre el que se rota. Cualquier rotación 3D puede representarse de esta forma.

Como representación en memoria son más compactos que las matrices, numéricamente más estables, implican menos cálculos, no tienen los problemas de usar 3 ángulos (Euler) para rotar sobre cada eje (véase gimbal-lock), y son más fáciles de combinar e interpolar. Pero, son mucho menos intuitivos y más difíciles de entender.

La operación $\mathbf{q}\mathbf{v}\mathbf{q}^{-1}$, siendo $\mathbf{q}$ un cuaternión unitario $\langle a, \mathbf{u} \rangle$, tiene por efecto producir un giro del vector $\mathbf{v}$ de un ángulo 2α alrededor del eje $\mathbf{u}$. La demostración se basa en descomponer $\mathbf{v}$ en una componente paralela a $\mathbf{u}$ y otra perpendicular, y aplicar propiedades algebraicas. El dibujo era para ilustrar la demostración, pero no nos detendremos en ella (es muy tediosa en el sentido de que son mucho términos y pasos, pero en realidad son todos pasos algebraicos relativamente simples).

Entonces, la aplicación importante es representar rotaciones/orientaciones, y poder además operar con ellos de forma "simple" y eficiente. En particular, interpolar rotaciones resulta más directo si las rotaciones están planteadas de esta forma. La parte escalar del cuaternión se puede interpolar con la interpolación afín, la parte vectorial con slerp.

¿QUÉ HAY QUE SABER?

- Definición del problema
- **Interpolación afín completa**
 - combinación afín, combinación convexa, convex-hull
 - poder calcular los pesos (con signo), e interpretarlos (isolineas)
- Otros mecanismos (bilineal, hiperbólica, slerp, cuaterniones):
 - solo ideas básicas (cuándo convendría utilizarla, cómo difieren de la afín, etc)
 - pero sin recordar ecuaciones (solo recordar la formulación de la bilineal)